

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Open Data Analysis

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO4	Assessment:	Tool 2– Open Data Analysis
Weightage:		Total Marks:	
CO4 Statement: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.			
Student Name:	Dharshini H	Reg. No.:	192324268
Section / Batch:	B.Tech AI and DS	Date of Submission:	19-08-2026
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I Dharshini H, 192324268 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

1. Select any **open government dataset** and perform data exploration using importing, table functions, filtering, sorting, grouping, correlation analysis, and outlier detection. Present the results using suitable charts and publish the final analysis on an open-source platform.

ANSWER:

```
CO4 > at2_1.py > ...
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # =====
6 # 1. IMPORT DATASET
7 # =====
8
9 df = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\india_population.csv")
10
11 print("=" * 60)
12 print("INDIAN POPULATION DATA ANALYSIS")
13 print("=" * 60)
14
15 # =====
16 # 2. BASIC TABLE FUNCTIONS
17 # =====
18
19 print("\n1. FIRST 5 RECORDS")
20 print(df.head())
21
22 print("\n2. LAST 5 RECORDS")
23 print(df.tail())
24
25 print("\n3. NUMBER OF ROWS AND COLUMNS")
26 print(df.shape)
27
28 print("\n4. COLUMN NAMES")
29 print(df.columns.tolist())
30
31 print("\n5. DATA TYPES")
32 print(df.dtypes)
33
34 print("\n6. DATASET INFORMATION")
35 df.info()
36
37
38 # =====
39 # 2. BASIC TABLE FUNCTIONS
40 # =====
41
42 print("\n1. FIRST 5 RECORDS")
43 print(df.head())
44
45 print("\n2. LAST 5 RECORDS")
46 print(df.tail())
47
48 print("\n3. NUMBER OF ROWS AND COLUMNS")
49 print(df.shape)
50
51 print("\n4. COLUMN NAMES")
52 print(df.columns.tolist())
53
54 print("\n5. DATA TYPES")
55 print(df.dtypes)
56
57 print("\n6. DATASET INFORMATION")
58 df.info()
59
60 print("\n7. STATISTICAL SUMMARY")
61 print(df.describe())
62
63 print("\n8. MISSING VALUES")
64 print(df.isnull().sum())
65
66 print("\n9. DUPLICATE ROWS")
67 print(df.duplicated().sum())
```

```

46 # =====
47 # 3. DATA CLEANING
48 # =====
49
50 # Remove duplicate records
51 df = df.drop_duplicates()
52
53 # Remove spaces from column names
54 df.columns = df.columns.str.strip()
55
56 # Convert numerical columns
57 numeric_columns = [
58     "Population",
59     "Male",
60     "Female",
61     "Rural",
62     "Urban",
63     "Area",
64     "Literacy"
65 ]
66
67 for column in numeric_columns:
68     df[column] = pd.to_numeric(df[column], errors="coerce")
69
70 # Remove rows containing missing values
71 df = df.dropna(subset=numeric_columns)
72
73 print("\nDATASET AFTER CLEANING")
74 print(df.shape)
75
76 # =====
77 # 4. FILTERING
78 # =====
79
80 print("\n" + "-" * 60)
81 print("FILTERING")
82 print("-" * 60)
83
84 # Population greater than 50 million
85 high_population = df[df["Population"] > 50000000]
86
87 print("\nStates with population greater than 50 million:")
88 print(
89     high_population[
90         ["State", "Population"]
91     ]
92 )
93
94 # Literacy greater than 80%
95 high_literacy = df[df["Literacy"] > 80]
96
97 print("\nStates with literacy greater than 80%:")
98 print(
99     high_literacy[
100         ["State", "Literacy"]
101     ]
102 )
103
104 # Urban population greater than 10 million
105 high_urban = df[df["Urban"] > 10000000]
106
107 print("\nStates with urban population greater than 10 million:")
108 print(
109     high_urban[
110         ["State", "Urban"]
111     ]
112 )
113
114 ...

```

```

114 # =====
115 # 5. SORTING
116 # =====
117
118 print("\n" + "=" * 60)
119 print("SORTING")
120 print("=" * 60)
121
122 # Sort by population
123 population_sorted = df.sort_values(
124     by="Population",
125     ascending=False
126 )
127 print("\nTop 10 states by population:")
128 print(
129     population_sorted[
130         ["State", "Population"]
131     ].head(10)
132 )
133 # Sort by literacy
134 literacy_sorted = df.sort_values(
135     by="Literacy",
136     ascending=False
137 )
138 print("\nTop 10 states by literacy:")
139 print(
140     literacy_sorted[
141         ["State", "Literacy"]
142     ].head(10)
143 )
144 # Sort by area
145 area_sorted = df.sort_values(
146     by="Area",
147     ascending=False
148 )
149 print("\nTop 10 states by area:")
150 print(
151     area_sorted[
152         ["State", "Area"]
153     ].head(10)
154 )
155 """
156 # =====
157 # 6. GROUPING
158 # =====
159
160 print("\n" + "=" * 60)
161 print("GROUPING")
162 print("=" * 60)
163
164 # Population category grouping
165 population_group = df.groupby(
166     "Population_Category",
167     observed=True
168 )["Population"].agg(
169     ["count", "mean", "sum"]
170 )
171
172 print("\nPopulation category summary:")
173 print(population_group)
174
175 # Gender population summary
176 gender_summary = df[
177     ["Male", "Female"]
178 ].sum()
179
180 print("\nTotal Male and Female Population:")
181 print(gender_summary)
182
183 # Rural and Urban summary
184 rural_urban_summary = df[
185     ["Rural", "Urban"]
186 ].sum()
187
188 print("\nTotal Rural and Urban Population:")
189 print(rural_urban_summary)
190 """

```

```

191 # =====
192 # 7. CORRELATION ANALYSIS
193 # =====
194 print("\n" + "=" * 60)
195 print("CORRELATION ANALYSIS")
196 print("=" * 60)
197 correlation = df[numeric_columns].corr()
198 print("\nCorrelation Matrix:")
199 print(correlation)
200 # Individual correlations
201 population_urban_corr = df[
202     "Population"
203 ].corr(
204     df["Urban"]
205 )
206 population_area_corr = df[
207     "Population"
208 ].corr(
209     df["Area"]
210 )
211 literacy_urban_corr = df[
212     "Literacy"
213 ].corr(
214     df["Urban"]
215 )
216 population_literacy_corr = df[
217     "Population"
218 ].corr(
219     df["Literacy"]
220 )
221 print("\nPopulation vs Urban Population:")
222 print(round(population_urban_corr, 3))
223 print("\nPopulation vs Area:")
224 print(round(population_area_corr, 3))
225 print("\nLiteracy vs Urban Population:")
226 print(round(literacy_urban_corr, 3))
227 print("\nPopulation vs Literacy:")
228 print(round(population_literacy_corr, 3))
229
230 # =====
231 # 8. OUTLIER DETECTION USING IQR
232 # =====
233 print("\n" + "=" * 60)
234 print("OUTLIER DETECTION")
235 print("=" * 60)
236 def detect_outliers(data, column):
237     Q1 = data[column].quantile(0.25)
238     Q3 = data[column].quantile(0.75)
239     IQR = Q3 - Q1
240     lower_limit = Q1 - (1.5 * IQR)
241     upper_limit = Q3 + (1.5 * IQR)
242     outliers = data[
243         (data[column] < lower_limit) |
244         (data[column] > upper_limit)]
245     print("\nColumn:", column)
246     print("Q1:", Q1)
247     print("Q3:", Q3)
248     print("IQR:", IQR)
249     print("Lower Limit:", lower_limit)
250     print("Upper Limit:", upper_limit)
251     print("\nOutliers:")
252     print(
253         outliers[["State", column]])
254     return outliers
255 # Population outliers
256 population_outliers = detect_outliers(
257     df,
258     "Population"
259 )
260 # Literacy outliers
261 literacy_outliers = detect_outliers(
262     df,
263     "Literacy"
264 )
265
266 # =====
267 # 9. VISUALIZATION 1
268 # TOP 10 STATES BY POPULATION
269 # =====
270
271 top10_population = df.sort_values(
272     "Population",
273     ascending=False
274 ).head(10)
275
276 plt.figure(figsize=(12, 6))
277
278 plt.bar(
279     top10_population["State"],
280     top10_population["Population"]
281 )
282
283 plt.title("Top 10 States by Population")
284 plt.xlabel("State")
285 plt.ylabel("Population")
286 plt.xticks(rotation=45)
287
288 plt.tight_layout()
289 plt.show()
290

```

```

291 # =====
292 # 10. VISUALIZATION 2
293 # TOP 10 STATES BY LITERACY
294 # =====
295
296 top10_literacy = df.sort_values(
297     "Literacy",
298     ascending=False
299 ).head(10)
300
301 plt.figure(figsize=(12, 6))
302
303 plt.bar(
304     top10_literacy["State"],
305     top10_literacy["Literacy"]
306 )
307
308 plt.title("Top 10 States by Literacy Rate")
309 plt.xlabel("State")
310 plt.ylabel("Literacy Rate (%)")
311 plt.xticks(rotation=45)
312
313 plt.tight_layout()
314 plt.show()
315
316 # =====
317 # 11. VISUALIZATION 3
318 # POPULATION VS AREA
319 # =====
320
321 plt.figure(figsize=(8, 6))
322
323 plt.scatter(
324     df["Area"],
325     df["Population"]
326 )
327
328 plt.title("Population vs Area")
329 plt.xlabel("Area (sq. km)")
330 plt.ylabel("Population")
331
332 plt.tight_layout()
333 plt.show()
334
335 # =====
336 # 12. VISUALIZATION 4
337 # POPULATION VS URBAN POPULATION
338 # =====
339
340 plt.figure(figsize=(8, 6))
341
342 plt.scatter(
343     df["Urban"],
344     df["Population"]
345 )
346
347 plt.title("Population vs Urban Population")
348 plt.xlabel("Urban Population")
349 plt.ylabel("Total Population")
350
351 plt.tight_layout()
352 plt.show()
353
354 # =====
355 # 13. VISUALIZATION 5
356 # LITERACY DISTRIBUTION
357 # =====
358
359 plt.figure(figsize=(8, 6))
360
361 plt.hist(
362     df["Literacy"],
363     bins=10
364 )
365
366 plt.title("Distribution of Literacy Rate")
367 plt.xlabel("Literacy Rate (%)")
368 plt.ylabel("Number of States")
369
370 plt.tight_layout()
371 plt.show()
372
373 # =====
374 # 14. VISUALIZATION 6
375 # POPULATION OUTLIER BOXPLOT
376 # =====
377
378 plt.figure(figsize=(8, 6))
379
380 plt.boxplot(
381     df["Population"]
382 )
383
384 plt.title("Population Outlier Detection")
385 plt.ylabel("Population")
386
387 plt.tight_layout()
388 plt.show()
389

```

```

390 # =====
391 # 15. VISUALIZATION 7
392 # RURAL VS URBAN POPULATION
393 # =====
394
395 total_rural = df["Rural"].sum()
396 total_urban = df["Urban"].sum()
397
398 plt.figure(figsize=(7, 5))
399
400 plt.bar(
401     ["Rural", "Urban"],
402     [total_rural, total_urban]
403 )
404
405 plt.title("Total Rural vs Urban Population")
406 plt.xlabel("Population Type")
407 plt.ylabel("Population")
408
409 plt.tight_layout()
410 plt.show()
411
412 # =====
413 # 16. SAVE CLEANED DATASET
414 # =====
415
416 df.to_csv(
417     "cleaned_india_population.csv",
418     index=False
419 )
420
421 print("\n" + "=" * 60)
422 print("ANALYSIS COMPLETED SUCCESSFULLY")
423 print("=" * 60)
424
425 print("\nCleaned dataset saved as:")
426 print("cleaned_india_population.csv")

```

OUTPUT:

```

PS C:\Users\dhars\Documents\Query_processing\lab_questions> & C:\Users\dhars\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dhars/Documents/Query_processing/lab_questions/t2_1.py"
=====
INDIAN POPULATION DATA ANALYSIS
=====

1. FIRST 5 RECORDS
   State  Population    Male   Female   Rural   Urban   Area  Literacy  Population_Category
0  Andhra Pradesh  49577103  25679962  23897141  14843385  34733718  162968  67.02      High
1  Arunachal Pradesh  1383727  711471   672256   313691   1070036  83743  65.38      Low
2      Assam  31205576  16192141  15013435  8256995  22948581  78438  72.19      High
3      Bihar  104099452  54562497  49536955  11763238  92336214  94163  61.80      High
4  Chhattisgarh  25545198  13119888  12425310  5936704  19608494  135192  70.28      High

2. LAST 5 RECORDS
   State  Population    Male   Female   Rural   Urban   Area  Literacy  Population_Category
31  Delhi  16787941  8832365  7955576  16368242  419699  1483  86.21      Medium
32  Jammu and Kashmir  12267032  6316528  5950504  3413915  8853117  42241  67.16      Medium
33  Ladakh  274289  139518  134771  27511  246778  59146  77.20      Low
34  Lakshadweep  64473  33522  30951  14081  50392  32  91.85      Low
35  Puducherry  1247953  633554  614399  852477  395476  490  85.85      Low

3. NUMBER OF ROWS AND COLUMNS
(36, 9)

4. COLUMN NAMES
['State', 'Population', 'Male', 'Female', 'Rural', 'Urban', 'Area', 'Literacy', 'Population_Category']

5. DATA TYPES
State          str
Population     int64
Male           int64
Female         int64
Rural          int64
Urban          int64
Area           int64
Literacy       float64
Population_Category  str
dtype: object

```

```
6. DATASET INFORMATION
<class 'pandas.DataFrame'>
RangeIndex: 36 entries, 0 to 35
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   State        36 non-null     str
1   Population    36 non-null     int64
2   Male         36 non-null     int64
3   Female       36 non-null     int64
4   Rural        36 non-null     int64
5   Urban        36 non-null     int64
6   Area         36 non-null     int64
7   Literacy     36 non-null     float64
8   Population_Category  36 non-null     str
dtypes: float64(1), int64(6), str(2)
memory usage: 3.2 KB

7. STATISTICAL SUMMARY
      Population      Male      Female      Rural      Urban      Area      Literacy
count  3.600000e+01  3.600000e+01  3.600000e+01  3.600000e+01  3.600000e+01  36.000000  36.000000
mean    3.363483e+07  1.734314e+07  1.629169e+07  1.063195e+07  2.300288e+07  87962.055556  77.253956
std     4.305171e+07  2.219929e+07  2.085725e+07  1.313091e+07  3.210789e+07  94968.028580   8.510002
min     6.447300e+04  3.352200e+04  3.095100e+04  1.408100e+04  2.902500e+04  32.000000  61.800000
25%    1.439840e+06  7.396942e+05  7.001462e+05  5.898842e+05  8.114458e+05  15055.750000  70.040000
50%    2.106970e+07  1.097613e+07  1.006715e+07  6.471100e+06  1.239724e+07  54578.000000  76.600000
75%    5.229275e+07  2.708352e+07  2.510800e+07  1.654161e+07  3.479247e+07  131343.000000  83.562500
max     1.998123e+08  1.030363e+08  9.677600e+07  5.081567e+07  1.553141e+08  342239.000000  94.000000

8. MISSING VALUES
State        0
Population   0
Male         0
Female       0
Rural        0
Urban        0
Area         0
Literacy     0
Population_Category  0
dtype: int64

9. DUPLICATE ROWS
0

DATASET AFTER CLEANING
(36, 9)
```

SORTING			FILTERING		
Top 10 states by population:			States with population greater than 50 million:		
	State	Population		State	Population
25	Uttar Pradesh	199812341	3	Bihar	104099452
13	Maharashtra	112374333	6	Gujarat	60439692
3	Bihar	104099452	10	Karnataka	61095297
27	West Bengal	91276115	12	Madhya Pradesh	72626809
12	Madhya Pradesh	72626809	13	Maharashtra	112374333
22	Tamil Nadu	72147030	20	Rajasthan	68548437
20	Rajasthan	68548437	22	Tamil Nadu	72147030
10	Karnataka	61095297	25	Uttar Pradesh	199812341
6	Gujarat	60439692	27	West Bengal	91276115
0	Andhra Pradesh	49577103			
Top 10 states by literacy:			States with literacy greater than 80%:		
	State	Literacy		State	Literacy
11	Kerala	94.00	5	Goa	88.70
34	Lakshadweep	91.85	8	Himachal Pradesh	82.80
16	Mizoram	91.33	11	Kerala	94.00
5	Goa	88.70	13	Maharashtra	82.34
24	Tripura	87.22	16	Mizoram	91.33
28	Andaman and Nicobar Islands	86.27	21	Sikkim	81.42
31	Delhi	86.21	22	Tamil Nadu	80.09
29	Chandigarh	86.05	24	Tripura	87.22
35	Puducherry	85.85	28	Andaman and Nicobar Islands	86.27
8	Himachal Pradesh	82.80	29	Chandigarh	86.05
Top 10 states by area:			31	Delhi	86.21
	State	Area	34	Lakshadweep	91.85
20	Rajasthan	342239	35	Puducherry	85.85
12	Madhya Pradesh	308252			
13	Maharashtra	307713	States with urban population greater than 10 million:		
25	Uttar Pradesh	240928		State	Urban
6	Gujarat	196244	0	Andhra Pradesh	34733718
10	Karnataka	191791	2	Assam	22948581
0	Andhra Pradesh	162968	3	Bihar	92336214
18	Odisha	155707	4	Chhattisgarh	19608494
4	Chhattisgarh	135192	6	Gujarat	34396229
22	Tamil Nadu	130060	7	Haryana	16473380
			9	Jharkhand	25054408
			10	Karnataka	37460746
			11	Kerala	15941372
			12	Madhya Pradesh	52560022
			13	Maharashtra	61558660
			18	Odisha	34968721
			19	Punjab	17342361
			20	Rajasthan	51486731
			22	Tamil Nadu	37227867
			23	Telangana	21394246
			25	Uttar Pradesh	155314133
			27	West Bengal	62186417

GROUPING

Population category summary:

	count	mean	sum
Population_Category			
High	18	6.362302e+07	1145214270
Low	14	1.402405e+06	19633667
Medium	4	1.150147e+07	46005867

Total Male and Female Population:

Male 624353057

Female 586500747

dtype: int64

Total Rural and Urban Population:

Rural 382750055

Urban 828103749

dtype: int64

CORRELATION ANALYSIS

Correlation Matrix:

	Population	Male	Female	Rural	Urban	Area	Literacy
Population	1.000000	0.999895	0.999881	0.880011	0.980954	0.738257	-0.419023
Male	0.999895	1.000000	0.999551	0.876134	0.982399	0.736662	-0.422814
Female	0.999881	0.999551	1.000000	0.883934	0.979190	0.739784	-0.414890
Rural	0.880011	0.876134	0.883934	1.000000	0.770996	0.727931	-0.186462
Urban	0.980954	0.982399	0.979190	0.770996	1.000000	0.692192	-0.485588
Area	0.738257	0.736662	0.739784	0.727931	0.692192	1.000000	-0.499638
Literacy	-0.419023	-0.422814	-0.414890	-0.186462	-0.485588	-0.499638	1.000000

Population vs Urban Population:

0.981

Population vs Area:

0.738

Literacy vs Urban Population:

-0.486

Population vs Literacy:

-0.419

OUTLIER DETECTION

Column: Population

Q1: 1439840.5

Q3: 52292750.25

IQR: 50852909.75

Lower Limit: -74839524.125

Upper Limit: 128572114.875

Outliers:

	State	Population
25	Uttar Pradesh	199812341

Column: Literacy

Q1: 70.03999999999999

Q3: 83.5625

IQR: 13.522500000000008

Lower Limit: 49.756249999999998

Upper Limit: 103.84625000000001

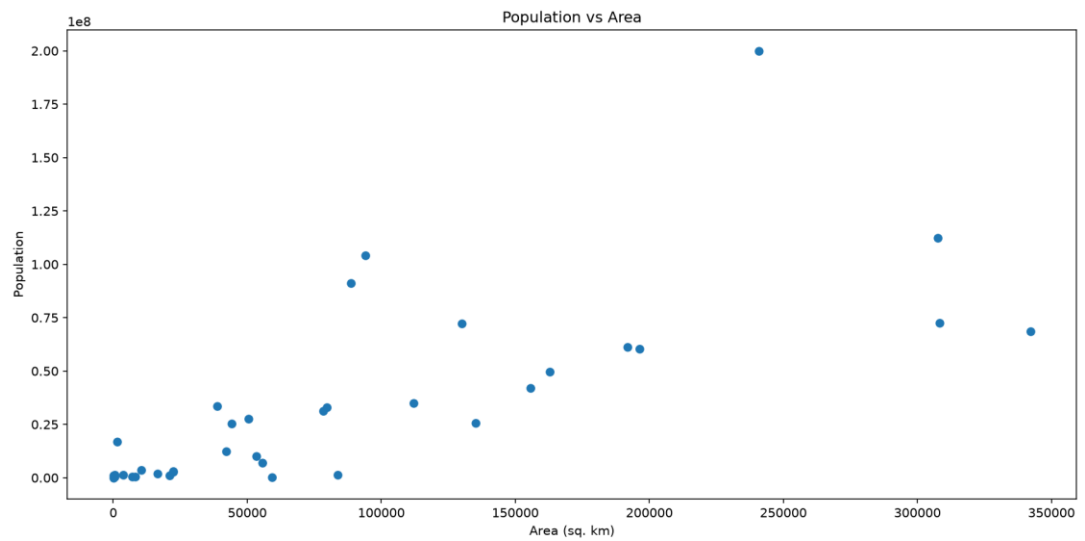
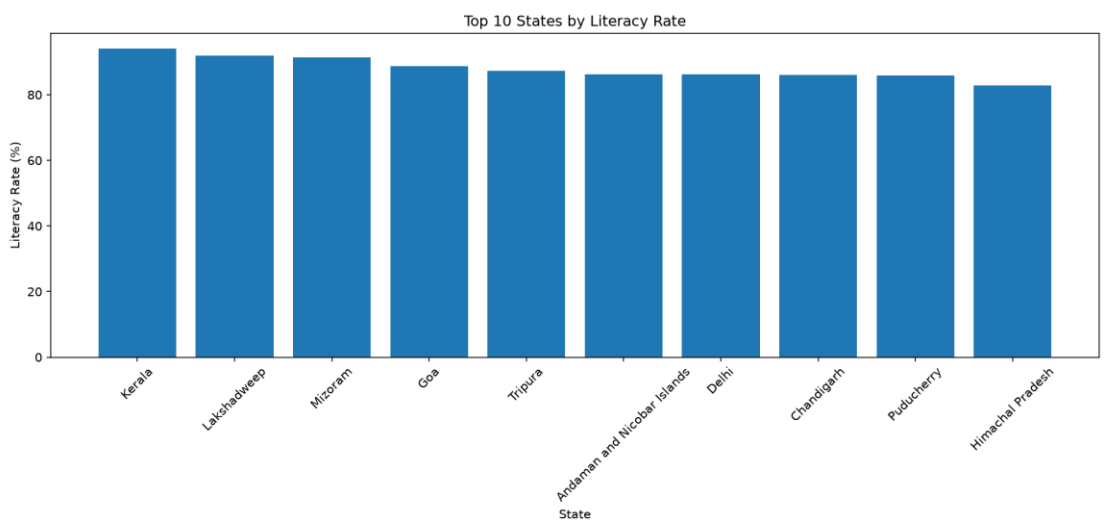
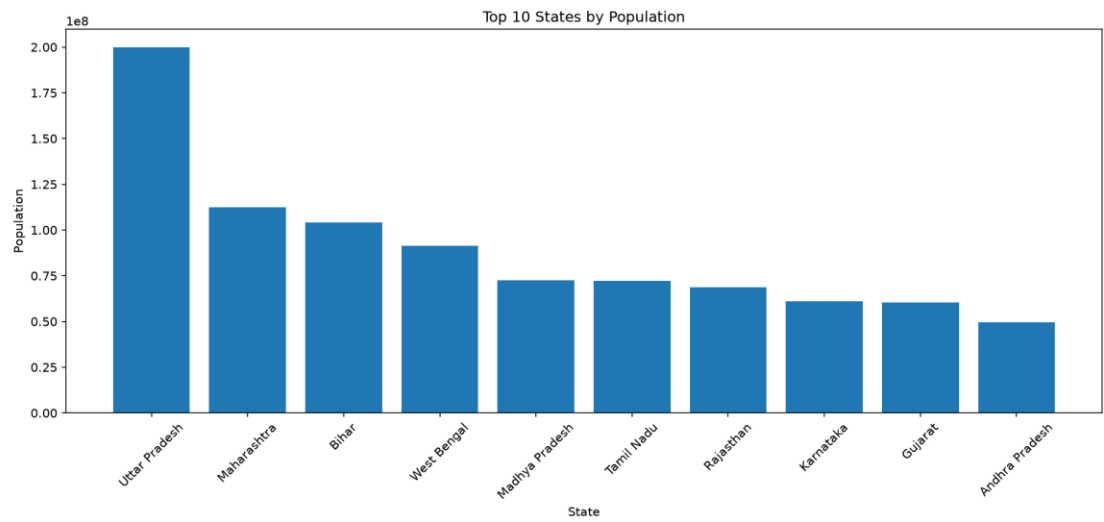
Outliers:

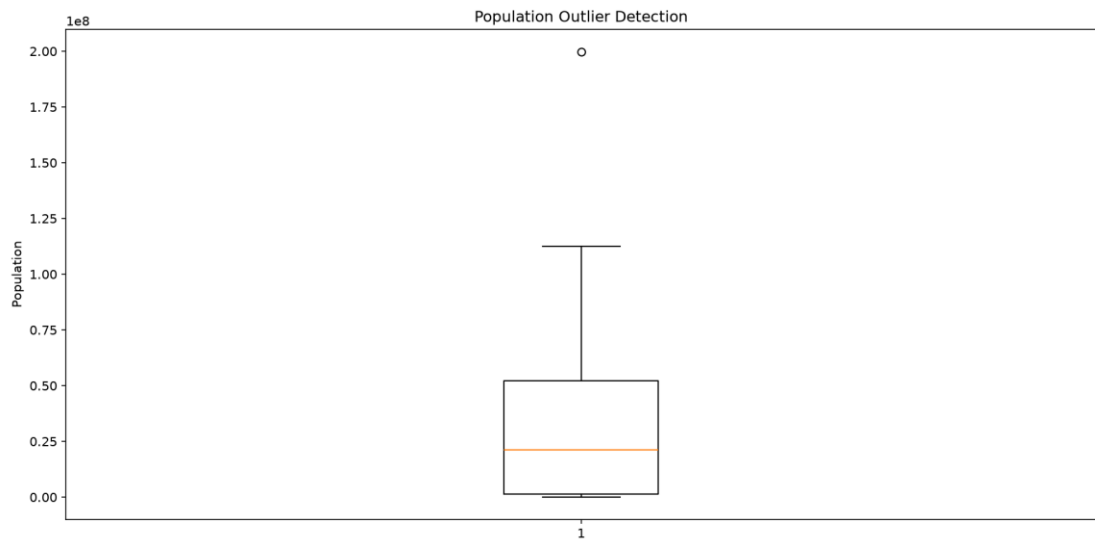
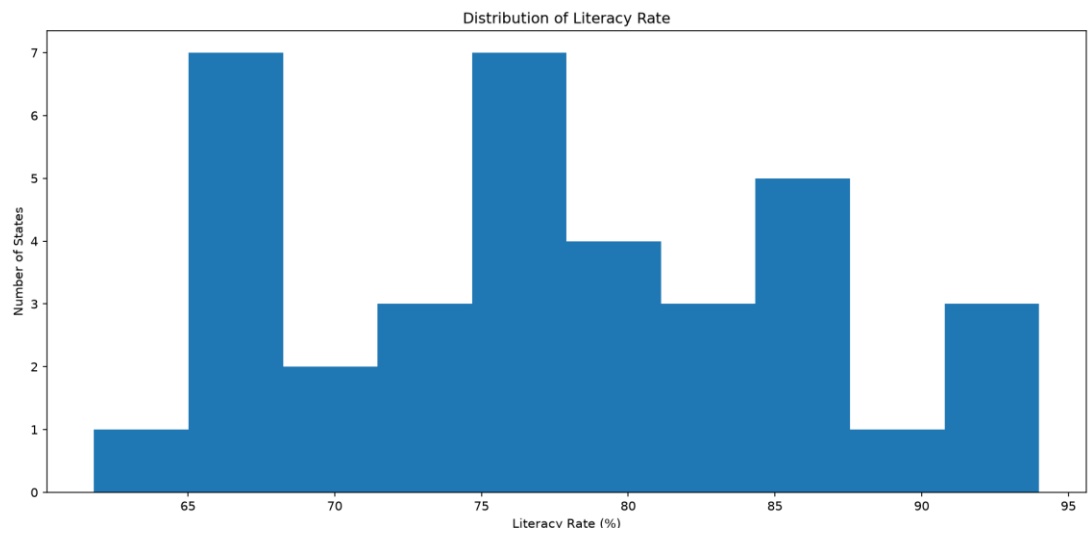
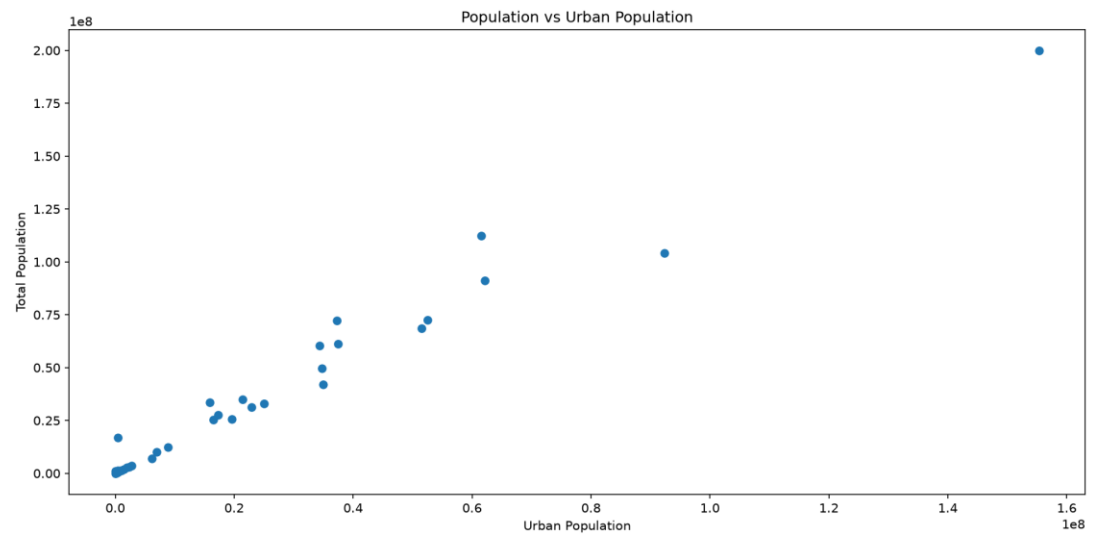
Empty DataFrame

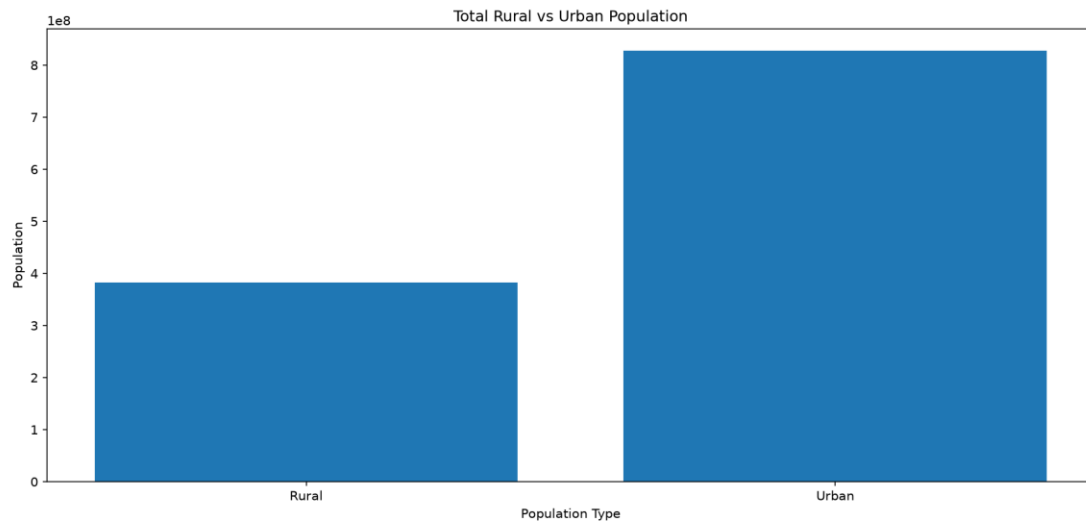
Columns: [State, Literacy]

Index: []

[]







2. Select any **open environmental dataset** such as air quality, rainfall, temperature, or pollution data. Analyze time-related patterns, identify correlations and outliers, create geographical visualizations/maps, and present the findings using an interactive visualization tool.

ANSWER:

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 # =====
5 # 1. IMPORT DATASET
6 # =====
7 df = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\C04\india_environmental_air_quality.csv")
8 print("-" * 70)
9 print("ENVIRONMENTAL AIR QUALITY DATA ANALYSIS")
10 print("-" * 70)
11 print("\nDataset loaded successfully!")
12 # Convert Date column
13 df["Date"] = pd.to_datetime(df["Date"])
14 # =====
15 # 2. BASIC DATA EXPLORATION
16 # =====
17 print("\n" + "-" * 70)
18 print("BASIC DATA EXPLORATION")
19 print("-" * 70)
20 print("\nFirst 5 records:")
21 print(df.head())
22 print("\nLast 5 records:")
23 print(df.tail())
24 print("\nDataset shape:")
25 print(df.shape)
26 print("\nColumn names:")
27 print(df.columns.tolist())
28 print("\nData types:")
29 print(df.dtypes)
30 print("\nStatistical summary:")
31 print(df.describe())
32 print("\nMissing values:")
33 print(df.isnull().sum())
34 print("\nDuplicate records:")
35 print(df.duplicated().sum())
36 --

```

```

36 # =====
37 # 3. CLEANING
38 # =====
39 df = df.drop_duplicates()
40 df = df.dropna(
41     subset=[
42         "Date",
43         "City",
44         "PM2_5",
45         "PM10",
46         "NO2",
47         "CO",
48         "Temperature_C",
49         "Rainfall_mm"
50     ]
51 )
52 print("\nDataset after cleaning:")
53 print(df.shape)
54 # =====
55 # 4. TIME-BASED ANALYSIS
56 # =====
57 print("\n" + "=" * 70)
58 print("TIME-BASED ANALYSIS")
59 print("-" * 70)
60 # Daily average PM2.5
61 daily_pm25 = df.groupby("Date")["PM2_5"].mean()
62 print("\nDaily average PM2.5:")
63 print(daily_pm25.head(10))
64 # Monthly average
65 monthly_pm25 = df.groupby(
66     ["Year", "Month"]
67 )["PM2_5"].mean().reset_index()
68 print("\nMonthly average PM2.5:")
69 print(monthly_pm25)
70 # Yearly average
71 yearly_pm25 = df.groupby(
72     ["Year"]
73 )["PM2_5"].mean()
74 print("\nYearly average PM2.5:")
75 print(yearly_pm25)
76 # =====
77 # 5. MONTHLY ANALYSIS
78 # =====
79 monthly_city = df.groupby(
80     ["Month", "Month_Name"]
81 )
82 [
83     ["PM2_5", "PM10", "NO2", "CO", "Temperature_C", "Rainfall_mm"]
84 ].mean().reset_index()
85 monthly_city = monthly_city.sort_values("Month")
86 print("\nAverage environmental values by month:")
87 print(monthly_city)
88 # =====
89 # 6. CITY-WISE ANALYSIS
90 # =====
91 print("\n" + "=" * 70)
92 print("CITY-WISE ANALYSIS")
93 print("-" * 70)
94 city_average = df.groupby("City")[
95     [
96         "PM2_5", "PM10", "NO2", "CO",
97         "Temperature_C", "Rainfall_mm"
98     ]
99 ].mean()
100 print("\nAverage values by city:")
101 print(city_average)
102 # =====
103 # 7. SORTING
104 # =====
105 print("\nCities with highest average PM2.5:")
106 top_cities = {
107     df.groupby("City")["PM2_5"]
108     .mean()
109     .sort_values(ascending=False)
110 }
111 print(top_cities)
112 # =====
113 # 8. CORRELATION ANALYSIS
114 # =====
115 print("\n" + "=" * 70)
116 print("CORRELATION ANALYSIS")
117 print("-" * 70)
118 numeric_columns = [
119     "PM2_5",
120     "PM10",
121     "NO2",
122     "CO",
123     "Temperature_C",
124     "Rainfall_mm"
125 ]
126 correlation = df[numeric_columns].corr()
127 print("\nCorrelation matrix:")
128 print(correlation)
129 print("\nPM2.5 vs PM10:")
130 print(
131     round(
132         df["PM2_5"].corr(df["PM10"]),
133         3)
134 )
135 print("\nPM2.5 vs NO2:")
136 print(
137     round(
138         df["PM2_5"].corr(df["NO2"]),
139         3)
140 )
141 print("\nPM2.5 vs Temperature:")
142 print(
143     round(
144         df["PM2_5"].corr(df["Temperature_C"]),
145         3)
146 )
147 print("\nPM2.5 vs Rainfall:")
148 print(
149     round(
150         df["PM2_5"].corr(df["Rainfall_mm"]),
151         3)
152 )
153 # =====

```

```

148 # -----
149 # 9. OUTLIER DETECTION
150 # -----
151 print("\n" + "-" * 70)
152 print("OUTLIER DETECTION")
153 print("-" * 70)
154 def detect_outliers(data, column):
155     Q1 = data[column].quantile(0.25)
156     Q3 = data[column].quantile(0.75)
157     IQR = Q3 - Q1
158     lower = Q1 - 1.5 * IQR
159     upper = Q3 + 1.5 * IQR
160     outliers = data[
161         (data[column] < lower) |
162         (data[column] > upper)]
163     print("\nColumn:", column)
164     print("Q1:", round(Q1, 2))
165     print("Q3:", round(Q3, 2))
166     print("IQR:", round(IQR, 2))
167     print("Lower limit:", round(lower, 2))
168     print("Upper limit:", round(upper, 2))
169     print("Number of outliers:", len(outliers))
170
171     return outliers
172 pm25_outliers = detect_outliers(
173     df,
174     "PM2_5")
175 pm10_outliers = detect_outliers(
176     df,
177     "PM10")
178 # Show PM2.5 outlier records
179 print("\nPM2.5 outlier records:")
180 print(
181     pm25_outliers[
182         [
183             "Date",
184             "City",
185             "PM2_5",
186             "PM10"]].head(20))
187
188 # -----
189 # 10. CHART 1
190 # MONTHLY PM2.5 TREND
191 # -----
192 plt.figure(figsize=(12, 6))
193
194 plt.plot(
195     monthly_pm25["Month"].astype(str),
196     monthly_pm25["PM2_5"],
197     marker="o"
198 )
199
200 plt.title("Monthly PM2.5 Trend")
201 plt.xlabel("Month")
202 plt.ylabel("Average PM2.5")
203
204 plt.grid(True)
205 plt.tight_layout()
206
207 plt.show()
208
209 # -----
210 # 11. CHART 2
211 # YEARLY PM2.5
212 # -----
213
214 plt.figure(figsize=(8, 5))
215
216 plt.bar(
217     yearly_pm25.index.astype(str),
218     yearly_pm25.values
219 )
220
221 plt.title("Yearly Average PM2.5")
222 plt.xlabel("Year")
223 plt.ylabel("Average PM2.5")
224
225 plt.tight_layout()
226
227 plt.show()
228
229 # -----
230 # 12. CHART 3
231 # CITY-WISE PM2.5
232 # -----
233 plt.figure(figsize=(12, 6))
234 city_pm25 = (
235     df.groupby("City")["PM2_5"]
236     .mean()
237     .sort_values(ascending=False)
238 )
239
240 plt.bar(
241     city_pm25.index,
242     city_pm25.values
243 )
244
245 plt.title("Average PM2.5 by City")
246 plt.xlabel("City")
247 plt.ylabel("Average PM2.5")
248 plt.xticks(rotation=45)
249 plt.tight_layout()
250 plt.show()
251
252 # -----
253 # 13. CHART 4
254 # PM2.5 VS PM10
255 # -----
256 plt.figure(figsize=(8, 6))
257
258 plt.scatter(
259     df["PM2_5"],
260     df["PM10"],
261     alpha=0.3
262 )
263
264 plt.title("PM2.5 vs PM10")
265 plt.xlabel("PM2.5")
266 plt.ylabel("PM10")
267 plt.tight_layout()
268 plt.show()
269
270 # -----

```

```

266 # 14. CHART 5 PM2.5 VS RAINFALL
267 # =====
268 plt.figure(figsize=(8, 6))
269 plt.scatter(df["Rainfall_mm"],
270            df["PM2_5"],
271            alpha=0.3)
272 plt.title("Rainfall vs PM2.5")
273 plt.xlabel("Rainfall (mm)")
274 plt.ylabel("PM2.5")
275 plt.tight_layout()
276 plt.show()
277 # =====
278 # 15. CHART 6 PM2.5 DISTRIBUTION
279 # =====
280 plt.figure(figsize=(8, 6))
281 plt.hist(df["PM2_5"],
282         bins=30)
283 plt.title("PM2.5 Distribution")
284 plt.xlabel("PM2.5")
285 plt.ylabel("Number of Observations")
286 plt.tight_layout()
287 plt.show()
288 # =====
289 # 16. CHART 7 PM2.5 BOXPLOT
290 # =====
291 plt.figure(figsize=(8, 6))
292 plt.boxplot(df["PM2_5"])
293 plt.title("PM2.5 Outlier Detection")
294 plt.ylabel("PM2.5")
295 plt.tight_layout()
296 plt.show()
297 # =====
298 # 17. CHART 8 RAINFALL TREND
299 # =====
300 monthly_rainfall = df.groupby("Month")["Rainfall_mm"].mean()
301 plt.figure(figsize=(10, 6))
302 plt.plot(monthly_rainfall.index, monthly_rainfall.values, marker="o")
303 plt.title("Monthly Rainfall Trend")
304 plt.xlabel("Month")
305 plt.ylabel("Average Rainfall (mm)")
306 plt.grid(True)
307 plt.tight_layout()
308 plt.show()

310 # =====
311 # 18. GEOGRAPHICAL VISUALIZATION
312 # =====
313 # City-level average pollution
314 geo_data = df.groupby(
315     ["City", "Latitude", "Longitude"]
316 )["PM2_5"].mean().reset_index()
317 plt.figure(figsize=(10, 8))
318 scatter = plt.scatter(
319     geo_data["Longitude"],
320     geo_data["Latitude"],
321     s=geo_data["PM2_5"] * 2,
322     c=geo_data["PM2_5"],
323     alpha=0.7
324 )
325 for i in range(len(geo_data)):
326     plt.annotate(
327         geo_data.iloc[i]["City"],
328         (
329             geo_data.iloc[i]["Longitude"],
330             geo_data.iloc[i]["Latitude"]
331         )
332     )
333 plt.colorbar(
334     scatter,
335     label="Average PM2.5"
336 )
337 plt.title(
338     "Geographical Distribution of Average PM2.5"
339 )
340 plt.xlabel("Longitude")
341 plt.ylabel("Latitude")
342 plt.grid(True)
343 plt.tight_layout()
344 plt.show()
345 # =====
346 # 19. SAVE RESULTS
347 # =====
348 city_average.to_csv(
349     "city_environmental_summary.csv"
350 )
351 monthly_city.to_csv(
352     "monthly_environmental_summary.csv",
353     index=False
354 )
355 pm25_outliers.to_csv(
356     "pm25_outliers.csv",
357     index=False
358 )
359
360 print("\n" + "-" * 70)
361 print("ANALYSIS COMPLETED")
362 print("-" * 70)
363
364 print("\nFiles generated:")
365
366 print("1. city_environmental_summary.csv")
367 print("2. monthly_environmental_summary.csv")
368 print("3. pm25_outliers.csv")

```

OUTPUT:

```
First 5 records:
      Date      City      Latitude      Longitude      PM2_5      PM10      NO2      CO      Temperature_C      Rainfall_mm      Year      Month      Month_Name
0 2023-01-01      Delhi      28.6139      77.2090      228.968557      418.576949      50.817167      1.136235      19.553340      12.058660      2023      1      Jan
1 2023-01-01      Mumbai      19.0760      72.8777      117.038460      211.377025      39.647032      0.910589      18.133422      22.706626      2023      1      Jan
2 2023-01-01      Kolkata      22.5726      88.3639      142.638818      275.307283      49.421022      0.913497      15.158494      16.968178      2023      1      Jan
3 2023-01-01      Chennai      13.0827      80.2707      117.432464      189.650458      57.080530      0.962019      15.203321      20.315318      2023      1      Jan
4 2023-01-01      Bengaluru      12.9716      77.5946      120.226443      252.211830      44.930275      0.922519      17.321880      10.171024      2023      1      Jan

Last 5 records:
      Date      City      Latitude      Longitude      PM2_5      PM10      NO2      CO      Temperature_C      Rainfall_mm      Year      Month      Month_Name
7305 2024-12-31      Hyderabad      17.3850      78.4867      112.858306      207.568655      37.067000      0.840644      18.937012      12.157029      2024      12      Dec
7306 2024-12-31      Ahmedabad      23.0225      72.5714      139.017584      196.946686      41.123109      1.003313      26.150652      24.685747      2024      12      Dec
7307 2024-12-31      Pune      18.5204      73.8567      124.239680      224.585113      65.197413      0.872661      18.946307      31.583225      2024      12      Dec
7308 2024-12-31      Jaipur      26.9124      75.7873      134.788335      275.902224      37.798965      0.867048      18.359380      8.713160      2024      12      Dec
7309 2024-12-31      Lucknow      26.8467      80.9462      202.356658      322.620406      63.853413      0.951334      21.487546      15.997160      2024      12      Dec

Dataset shape:
(7310, 13)

Column names:
['Date', 'City', 'Latitude', 'Longitude', 'PM2_5', 'PM10', 'NO2', 'CO', 'Temperature_C', 'Rainfall_mm', 'Year', 'Month', 'Month_Name']

Data types:
Date      datetime64[us]
City      str
Latitude      float64
Longitude      float64
PM2_5      float64
PM10      float64
NO2      float64
CO      float64
Temperature_C      float64
Rainfall_mm      float64
Year      int64
Month      int64
Month_Name      str
dtype: object

Statistical summary:
      Date      Latitude      Longitude      PM2_5      PM10      ...      CO      Temperature_C      Rainfall_mm      Year      Month
count      7310      7310.000000      7310.000000      7310.000000      7310.000000      ...      7310.000000      7310.000000      7310.000000      7310.000000
mean      2024-01-01 00:00:00      20.900380      77.796420      98.794568      173.020080      ...      0.745487      26.430636      89.577138      2023.500684      6.519836
min      2023-01-01 00:00:00      12.971600      72.571400      5.000000      10.000000      ...      0.219911      8.971061      6.001061      2023.000000      1.000000
25%      2023-07-02 00:00:00      17.385000      73.856700      61.789932      108.557224      ...      0.629282      22.277009      20.387212      2023.000000      4.000000
50%      2024-01-01 00:00:00      20.824300      77.401800      95.194413      167.409700      ...      0.738513      27.016833      38.715440      2024.000000      7.000000
75%      2024-07-02 00:00:00      26.846700      80.270700      129.961137      231.221343      ...      0.856268      30.567507      140.300790      2024.000000      10.000000
max      2024-12-31 00:00:00      28.613900      88.363900      486.977426      775.284140      ...      1.346881      41.493308      415.619561      2024.000000      12.000000
std      NaN      5.326089      4.468299      47.991739      86.677118      ...      0.161366      5.547306      94.904616      0.500034      3.449787
```

```
[8 rows x 11 columns]

Missing values:
Date      0
City      0
Latitude      0
Longitude      0
PM2_5      0
PM10      0
NO2      0
CO      0
Temperature_C      0
Rainfall_mm      0
Year      0
Month      0
Month_Name      0
dtype: int64

Duplicate records:
0

Dataset after cleaning:
(7310, 13)

=====
TIME-BASED ANALYSIS
=====

Daily average PM2.5:
Date
2023-01-01      144.141866
2023-01-02      144.011150
2023-01-03      139.138490
2023-01-04      132.954789
2023-01-05      140.907161
2023-01-06      141.564812
2023-01-07      146.879887
2023-01-08      144.454457
2023-01-09      136.029464
2023-01-10      142.659746
Name: PM2_5, dtype: float64

Monthly average PM2.5:
      Year      Month      PM2_5
0 2023      1      142.392548
1 2023      2      140.621410
2 2023      3      85.694657
3 2023      4      85.485719
4 2023      5      86.259214
5 2023      6      68.268241
6 2023      7      69.435384
7 2023      8      69.687267
8 2023      9      69.052368
9 2023      10      83.601763
10 2023      11      138.823715
11 2023      12      141.840595
12 2024      1      140.134913
13 2024      2      141.440901
14 2024      3      86.258760
15 2024      4      89.172986
16 2024      5      86.022941
17 2024      6      68.640901
18 2024      7      69.779307
19 2024      8      70.980636
20 2024      9      69.307450
21 2024      10      86.387523
22 2024      11      144.530906
23 2024      12      142.190154

Yearly average PM2.5:
Year
2023      98.171376
2024      99.416057
Name: PM2_5, dtype: float64

Average environmental values by month:
      Month      Month_Name      PM2_5      PM10      NO2      CO      Temperature_C      Rainfall_mm
0 1      Jan      141.263730      248.314793      47.545149      0.871200      18.020607      19.930278
1 2      Feb      141.042963      247.089909      47.581025      0.870713      20.984595      14.659049
2 3      Mar      85.976708      150.717995      38.013177      0.704145      26.118899      17.975003
3 4      Apr      87.329352      153.111824      39.216889      0.711114      30.921475      25.370378
4 5      May      86.141078      151.418087      39.407323      0.708090      34.132964      43.650244
5 6      Jun      68.454571      119.992438      36.251937      0.650486      32.012157      152.132714
6 7      Jul      69.607346      123.022269      36.117094      0.658216      29.019170      255.245446
7 8      Aug      70.336452      122.326932      36.696731      0.661756      27.019269      221.813914
8 9      Sep      69.179909      118.391843      35.818340      0.657380      28.089805      173.835200
9 10      Oct      84.994643      149.828332      38.789325      0.704187      26.933332      90.634439
10 11      Nov      141.677311      247.220479      47.215689      0.873888      22.994418      34.828786
11 12      Dec      142.035329      249.058118      47.074424      0.875187      19.078363      19.513533
```



```
=====
CITY-WISE ANALYSIS
=====

Average values by city:
      PM2_5      PM10      NO2      CO      Temperature_C      Rainfall_mm
City
Ahmedabad  98.434317  173.299478  40.397882  0.738261  26.364751  92.156977
Bengaluru  58.270688  103.047861  34.259658  0.623727  26.414174  91.271485
Chennai    73.152932  128.602775  37.019972  0.666129  26.437247  90.646599
Delhi      161.478143  282.686290  50.846627  0.928760  26.451734  90.182046
Hyderabad  78.423482  137.280217  37.939095  0.688261  26.391646  92.127786
Jaipur     107.471359  187.270548  41.860235  0.772634  26.436291  86.114912
Kolkata    117.007796  202.929638  43.756026  0.799480  26.409912  87.686293
Lucknow    137.644454  241.039088  47.133733  0.865671  26.497362  88.414093
Mumbai     86.927188  152.849041  39.138792  0.715162  26.392239  88.927357
Pune       69.135322  121.195868  36.100696  0.656788  26.511002  88.243987

Cities with highest average PM2.5:
City
Delhi      161.478143
Lucknow    137.644454
Kolkata    117.007796
Jaipur     107.471359
Ahmedabad  98.434317
Mumbai     86.927188
Hyderabad  78.423482
Chennai    73.152932
Pune       69.135322
Bengaluru  58.270688
Name: PM2_5, dtype: float64

-----
CORRELATION ANALYSIS
-----

Correlation matrix:
      PM2_5      PM10      NO2      CO      Temperature_C      Rainfall_mm
PM2_5      1.000000  0.956534  0.718342  0.853612  -0.489216  -0.409176
PM10      0.956534  1.000000  0.691418  0.821958  -0.476345  -0.399852
NO2       0.718342  0.691418  1.000000  0.631145  -0.352403  -0.298905
CO        0.853612  0.821958  0.631145  1.000000  -0.428631  -0.362336
Temperature_C -0.489216 -0.476345 -0.352403 -0.428631  1.000000  0.352605
Rainfall_mm -0.409176 -0.399852 -0.298905 -0.362336  0.352605  1.000000

PM2.5 vs PM10:
0.957

=====
OUTLIER DETECTION
=====

Column: PM2_5
Q1: 61.79
Q3: 129.96
IQR: 68.17
Lower limit: -40.47
Upper limit: 232.22
Number of outliers: 32

Column: PM10
Q1: 108.56
Q3: 231.22
IQR: 122.66
Lower limit: -75.44
Upper limit: 415.22
Number of outliers: 27

PM2.5 outlier records:
      Date      City      PM2_5      PM10
129  2023-01-13  Lucknow  486.977426  775.284140
270  2023-01-28  Delhi    241.243390  420.110973
396  2023-02-09  Ahmedabad  362.722991  551.170368
480  2023-02-18  Delhi    235.479186  381.821970
846  2023-03-26  Ahmedabad  235.543194  258.867267
882  2023-03-30  Kolkata  239.274331  372.173101
1380 2023-05-19  Delhi    319.349055  471.762856
3410 2023-12-08  Delhi    250.996404  394.873512
3540 2023-12-21  Delhi    233.494066  439.527287
3760 2024-01-12  Delhi    246.853202  399.489954
3810 2024-01-17  Delhi    239.798951  450.324772
3990 2024-02-04  Delhi    233.367040  389.554647
4020 2024-02-07  Delhi    237.345561  384.519748
4040 2024-02-09  Delhi    262.135987  468.104277
4150 2024-02-20  Delhi    233.688214  444.421123
4210 2024-02-26  Delhi    237.422173  419.862949
4736 2024-04-18  Ahmedabad  246.275107  376.090936
5126 2024-05-27  Ahmedabad  265.766399  411.454489
5359 2024-06-19  Lucknow   335.850287  410.213988
6403 2024-10-02  Chennai   234.304564  316.548037
[]
```

